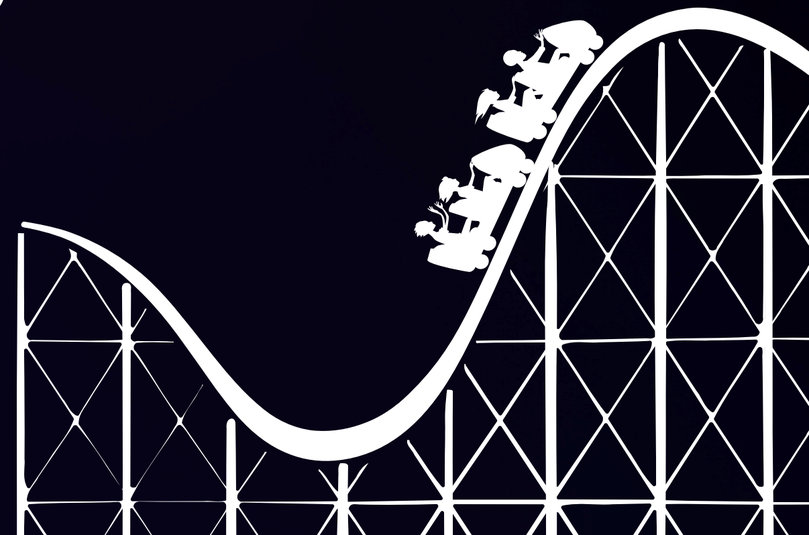
DATATHON WORKSHOP #1

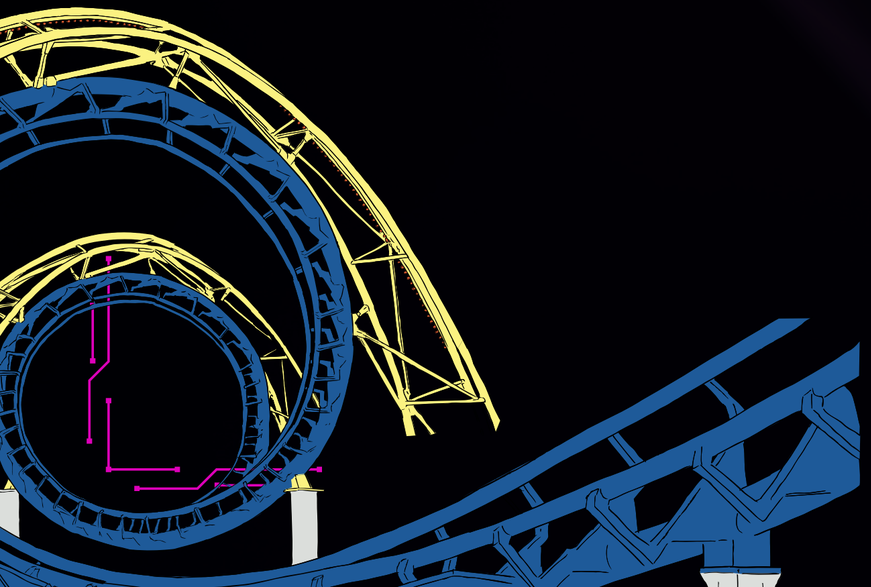
THE WAIT IS OVER: DATA SCIENCE FOR THEME PARK QUEUES



Before We Start... Let's Get to Know You!

Wooclap Code

NABIKOC



QUICK POLL — BEFORE WE LOOK AT ANY DATA

Slido Code: 2502834

We'll come back to this at the end and see who was right!

What do you think is the number one driving factor that causes a ride's wait time to skyrocket?

- A high overall park crowd density (park_crowd_index)
- Whether a parade or fireworks show is happening (special_event)
- What the queue looked like 30 minutes ago (previous_wait_minutes)
- The outdoor weather and temperature changing (temperature_c / rainfall_mm)

Your Canva profile name won't be shared

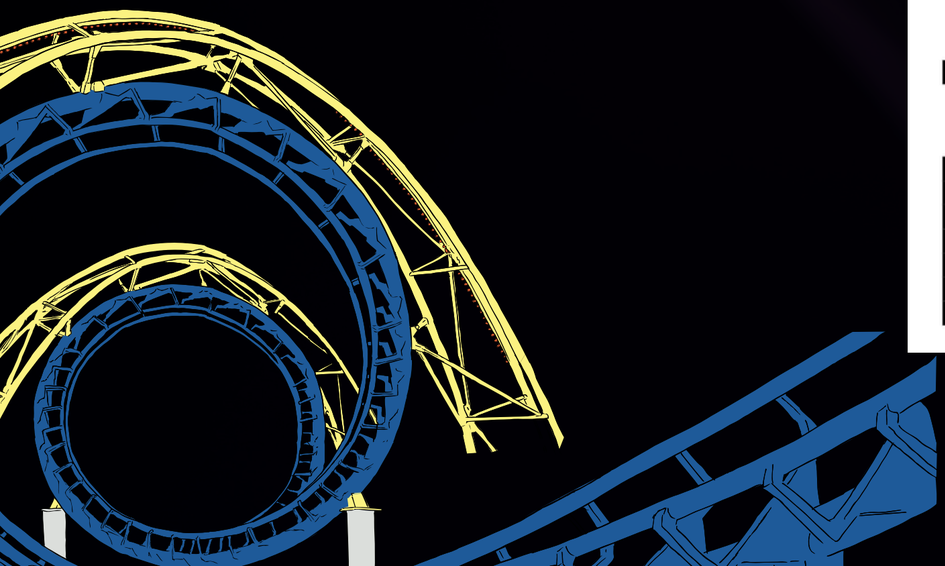
THE CHALLENGE: PREDICTING PARK WAIT TIMES

Imagine you are the Data Science team for a major theme park. Guests are frustrated by long lines, and the park wants to display an **accurate, real-time wait-time prediction for every ride, every 30 minutes**.

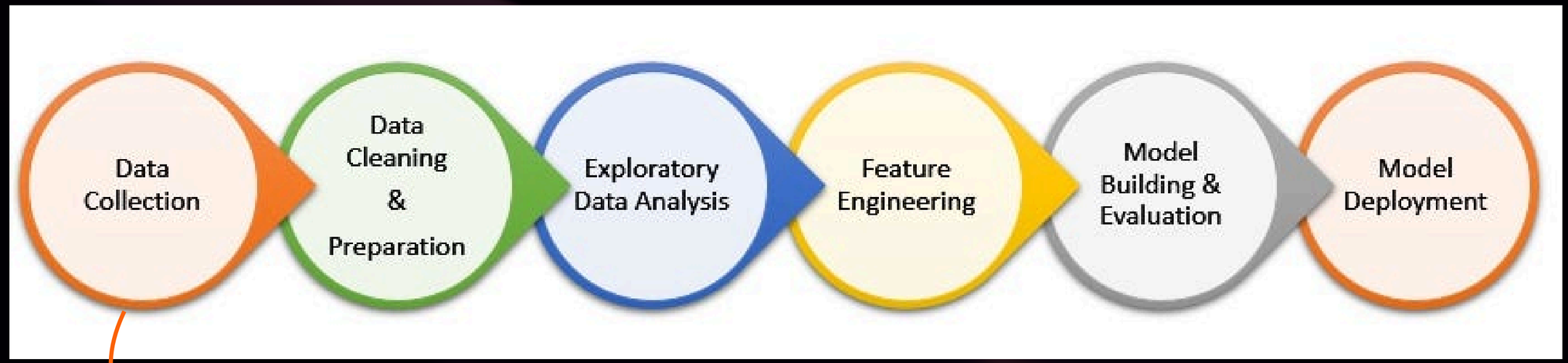
Your mission is to build a **Regression model** that takes live operational, environmental, and time data and predicts the exact number of minutes a guest will wait in line.

Scan the QR Code to access your Google Colab Notebooks

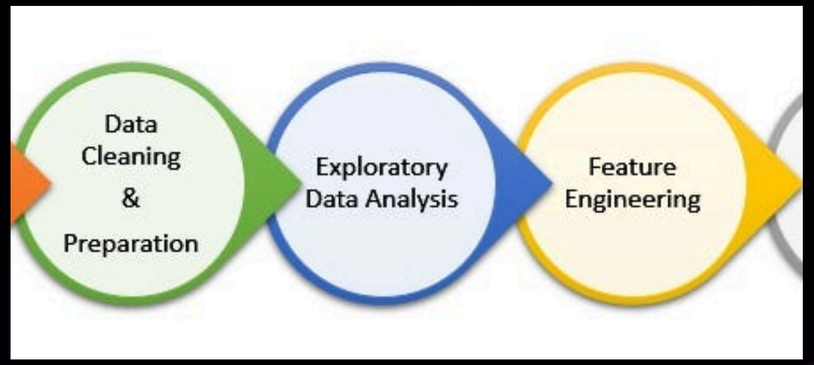
Please download a copy of this notebook and work on it locally



STEPS WE WILL BE COVERING TODAY



We already have the dataset collected for you, link in the colab nb!!



STEP 1: EXPLORING THE DATASET

NEVER SKIP STRAIGHT TO MODELING. LET'S INSPECT OUR PARK LOGS WITH PANDAS, CHECK FOR HIDDEN TRENDS, AND UNCOVER WHICH OPERATIONAL VARIABLES ACTUALLY DRIVE OUR WAIT TIMES.

EXPLORING THE THEME PARK DATASET

id	timestamp	ride_name	ride_type	capacity_per_hour	temperature_c	humidity_pct	rainfall_mm	park_crowd_index	special_event	ride_downtime_minutes_last_hour	previous_wait_minutes
TP002113	2026-09-12T	Dragon Coa	Roller Coaste	1100	29.1	77.9	0	63	Yes	0	48.9
TP002114	2026-09-12T	Sky Drop	Thrill Ride	900	29.1	77.9	0	63	Yes	0	44.5
TP002115	2026-09-12T	Pirate Voya	Water Ride	1250	29.1	77.9	0	63	Yes	0	34.6
TP002116	2026-09-12T	Jungle Rapic	Water Ride	1000	29.1	77.9	0	63	Yes	0	48.8
TP002117	2026-09-12T	Galaxy Spin	Thrill Ride	1300	29.1	77.9	0	63	Yes	0	36.2
TP002118	2026-09-12T	Magic Caro	Family Ride	1800	29.1	77.9	0	63	Yes	0	21.5
TP002119	2026-09-12T	Dino Advent	Family Ride	1450	29.1	77.9	0	63	Yes	0	23.9
TP002120	2026-09-12T	Haunted Ma	Dark Ride	1200	29.1	77.9	0	63	Yes	0	34.7
TP002121	2026-09-12T	Dragon Coa	Roller Coaste	1100	30	77.2	0	56.9	Yes	0	67.3
TP002122	2026-09-12T	Sky Drop	Thrill Ride	900	30	77.2	0	56.9	Yes	0	68
TP002123	2026-09-12T	Pirate Voya	Water Ride	1250	30	77.2	0	56.9	Yes	0	59.5
TP002124	2026-09-12T	Jungle Rapic	Water Ride	1000	30	77.2	0	56.9	Yes	0	68.3
TP002125	2026-09-12T	Galaxy Spin	Thrill Ride	1300	30	77.2	0	56.9	Yes	0	49
TP002126	2026-09-12T	Magic Caro	Family Ride	1800	30	77.2	0	56.9	Yes	0	41.2
TP002127	2026-09-12T	Dino Advent	Family Ride	1450	30	77.2	0	56.9	Yes	0	43.1
TP002128	2026-09-12T	Haunted Ma	Dark Ride	1200	30	77.2	0	56.9	Yes	0	50.2
TP002129	2026-09-12T	Dragon Coa	Roller Coaste	1100	30.7	76.6	0	58.3	Yes	0	70.6
TP002130	2026-09-12T	Sky Drop	Thrill Ride	900	30.7	76.6	0	58.3	Yes	0	78.2
TP002131	2026-09-12T	Pirate Voya	Water Ride	1250	30.7	76.6	0	58.3	Yes	0	61.3
TP002132	2026-09-12T	Jungle Rapic	Water Ride	1000	30.7	76.6	0	58.3	Yes	0	72
TP002133	2026-09-12T	Galaxy Spin	Thrill Ride	1300	30.7	76.6	0	58.3	Yes	0	53.6

These are our features from the csv file, we will be working with these in our colab notebooks

KEY FEATURES - WHAT DOES EACH SIGNIFY

Column	Meaning
`id`	unique row label
`timestamp`	date and time of the observation (every 30 minutes)
`ride_name`, `ride_type`	which ride, and its category
`capacity_per_hour`	how many guests the ride can handle per hour
`temperature_c`, `humidity_pct`, `rainfall_mm`	weather
`park_crowd_index`	how busy the park is overall (0–100)
`special_event`	is a park-wide special event on?
`ride_downtime_minutes_last_hour`	minutes the ride was broken down recently
`previous_wait_minutes`	the queue time at the previous half hour (a **lag feature**)
`wait_time_minutes`	**TARGET** — what we want to predict

LOADING & INSPECTING WITH PANDAS

```
import pandas as pd
```

Primary library used for data inspection

```
df = pd.read_csv("theme_park_train.csv")
df.head()
```

Load the dataset

Inspect the data

```
df.info()
```

Confirms the shape (2112 rows) and flags that timestamp is currently a raw text string that needs sanitizing.

```
df.describe()
```

Reveals averages and extremes (min/max), flagging highly skewed events like rainfall_mm and ride_downtime_minutes_last_hour

```
df.isna().sum()
```

Scans every single column for missing data entries or NaN holes 0 missing values found here!

LOADING & INSPECTING WITH PANDAS

```
df.isna().sum()
```

```
id                                0
timestamp                         0
ride_name                         0
ride_type                         0
capacity_per_hour                 0
temperature_c                     0
humidity_pct                      0
rainfall_mm                       0
park_crowd_index                  0
special_event                     0
ride_downtime_minutes_last_hour   0
previous_wait_minutes             0
wait_time_minutes                 0
dtype: int64
```

Inspect
the data

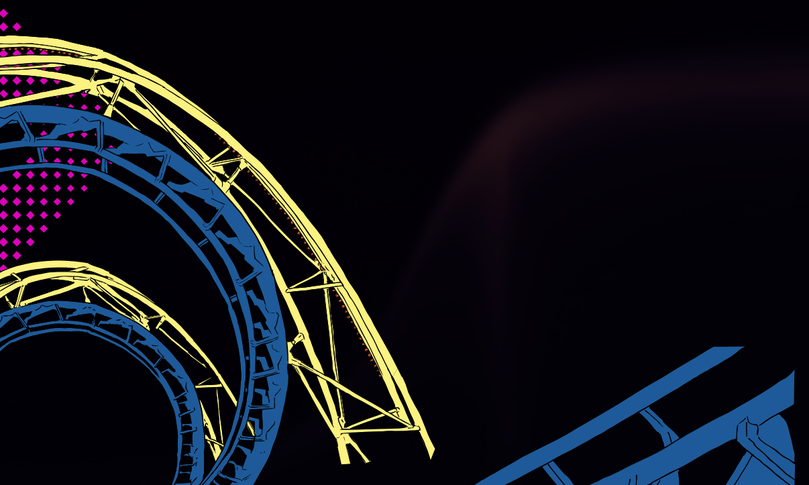
```
df.info()
```

```
(2112, 13)
<class 'pandas.DataFrame'>
RangeIndex: 2112 entries, 0 to 2111
Data columns (total 13 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   id                              2112 non-null   str
 1   timestamp                       2112 non-null   str
 2   ride_name                       2112 non-null   str
 3   ride_type                       2112 non-null   str
 4   capacity_per_hour               2112 non-null   int64
 5   temperature_c                   2112 non-null   float64
 6   humidity_pct                    2112 non-null   float64
 7   rainfall_mm                     2112 non-null   float64
 8   park_crowd_index                2112 non-null   float64
 9   special_event                   2112 non-null   str
 10  ride_downtime_minutes_last_hour 2112 non-null   int64
 11  previous_wait_minutes           2112 non-null   float64
 12  wait_time_minutes               2112 non-null   float64
dtypes: float64(6), int64(2), str(5)
memory usage: 214.6 KB
```

```
df.describe()
```

	capacity_per_hour	temperature_c	humidity_pct	rainfall_mm	park_crowd_index	ride_downtime_minutes_last_hour	previous_wait_minutes	wait_time_minutes
count	2112.000000	2112.000000	2112.000000	2112.000000	2112.000000	2112.000000	2112.000000	2112.000000
mean	1250.000000	30.924621	76.925379	0.196591	52.101894	0.764678	46.187737	46.307576
std	263.453722	1.591510	2.992769	0.718193	11.885060	3.950190	13.800210	13.590732
min	900.000000	27.300000	68.000000	0.000000	26.000000	0.000000	5.000000	9.500000
25%	1075.000000	29.800000	74.975000	0.000000	43.450000	0.000000	37.300000	37.400000
50%	1225.000000	30.800000	76.950000	0.000000	53.800000	0.000000	45.800000	45.900000
75%	1337.500000	32.100000	78.900000	0.000000	59.525000	0.000000	54.600000	54.600000
max	1800.000000	34.500000	84.500000	5.700000	86.000000	30.000000	94.200000	94.200000

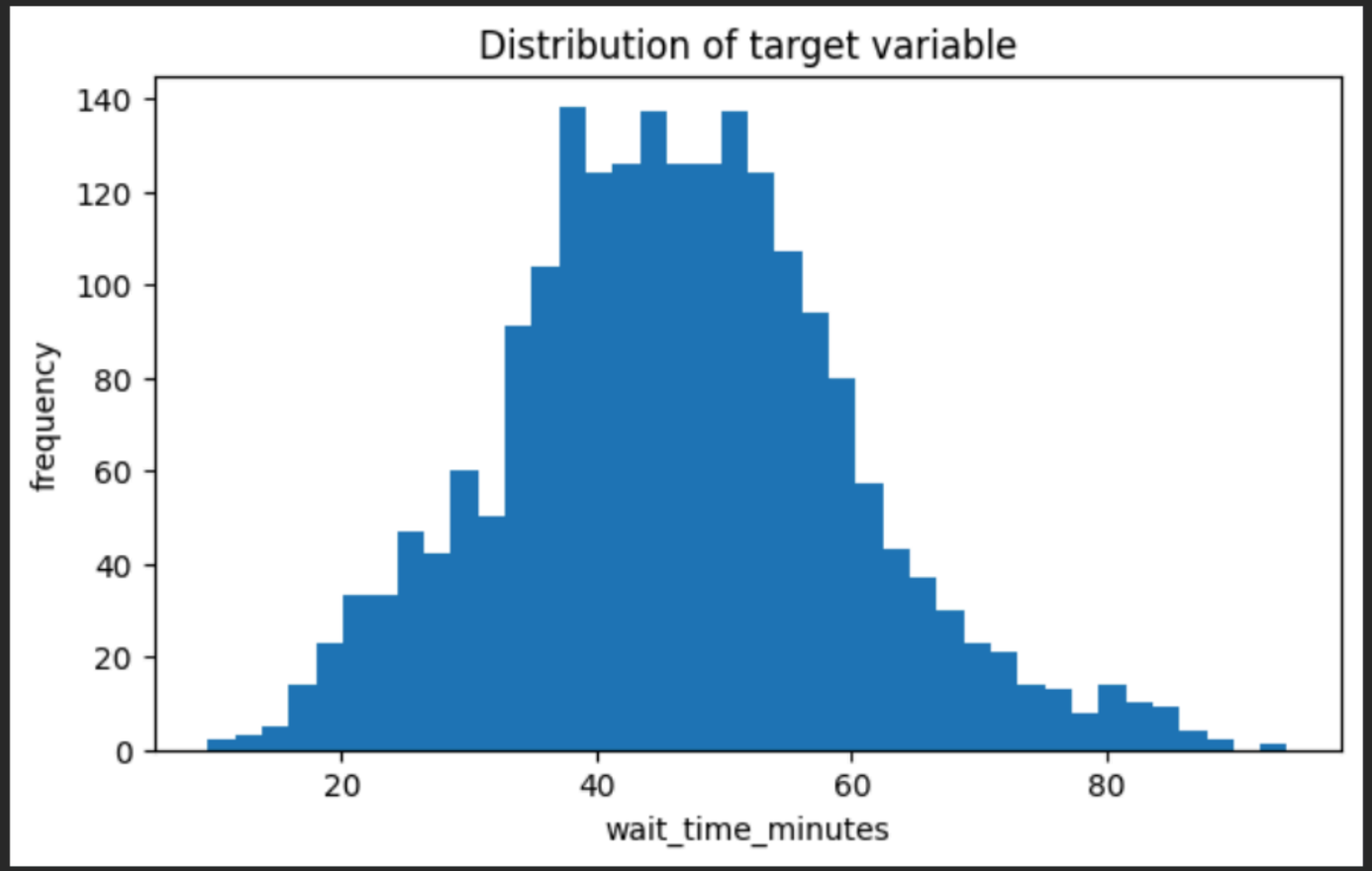
NOW COMES VISUALIZING THE CORRELATION/DEPENDENCY BETWEEN OUR TARGET FEATURE AND THE DIFFERENT DATA FEATURES



```
================================================================================
  THE EDA BLUEPRINT: HOW TO DECIDE WHAT TO GRAPH IN A HACKATHON
================================================================================

   [ STEP 1: TARGET PROFILE ] ──> Is your Target (Y) skewed? Are there outliers?
    Use a HISTOGRAM (plt.hist)
    Why: See the raw shape of the values you are trying to predict.

                                      |
                                      ▼

   [ STEP 2: NUMERIC VS TARGET ] ──> Does Y rise/fall smoothly across a continuous scale?
    Use a LINE PLOT (.plot) or SCATTER PLOT (plt.scatter)
    Why: Maps smooth numerical movements, rolling cycles, or linear paths (e.g., Time).

                                      |
                                      ▼

   [ STEP 3: CATEGORY VS TARGET ] ──> Do distinct group buckets structurally shift Y?
    Use a BOX PLOT (df.boxplot) or BAR CHART (kind='bar')
    Why: Contrasts average benchmarks and flags hidden outlier clusters across category words.

================================================================================
  NOTE : You do NOT graph every single feature column!
 Focus ONLY on columns that show a clear correlation signal or categorical group split.
================================================================================
```

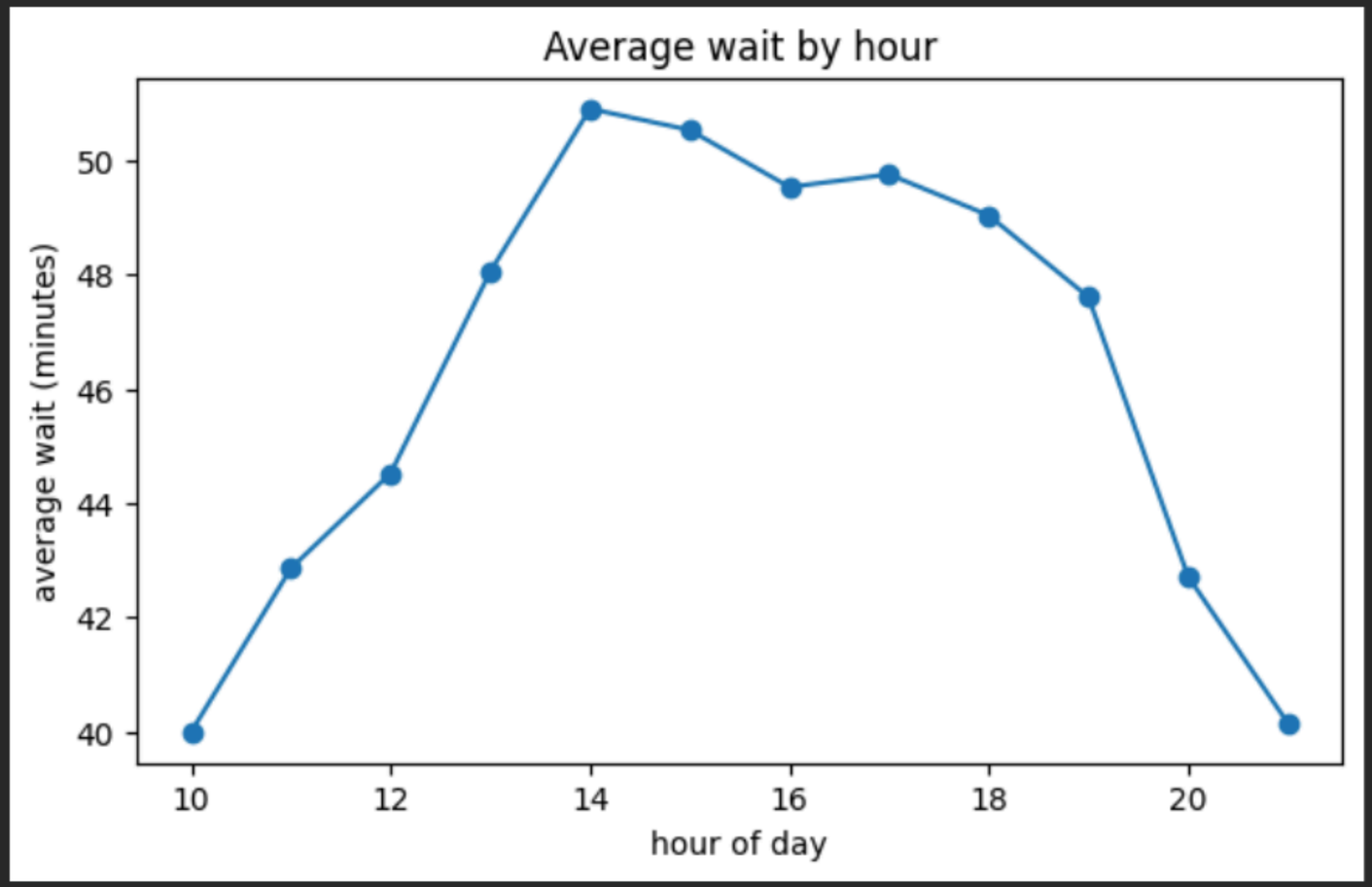
wait time with frequency of people

```
plt.figure(figsize=(7, 4))
plt.hist(df["wait_time_minutes"], bins=40)
plt.xlabel("wait_time_minutes")
plt.ylabel("frequency")
plt.title("Distribution of target variable")
plt.show()
```



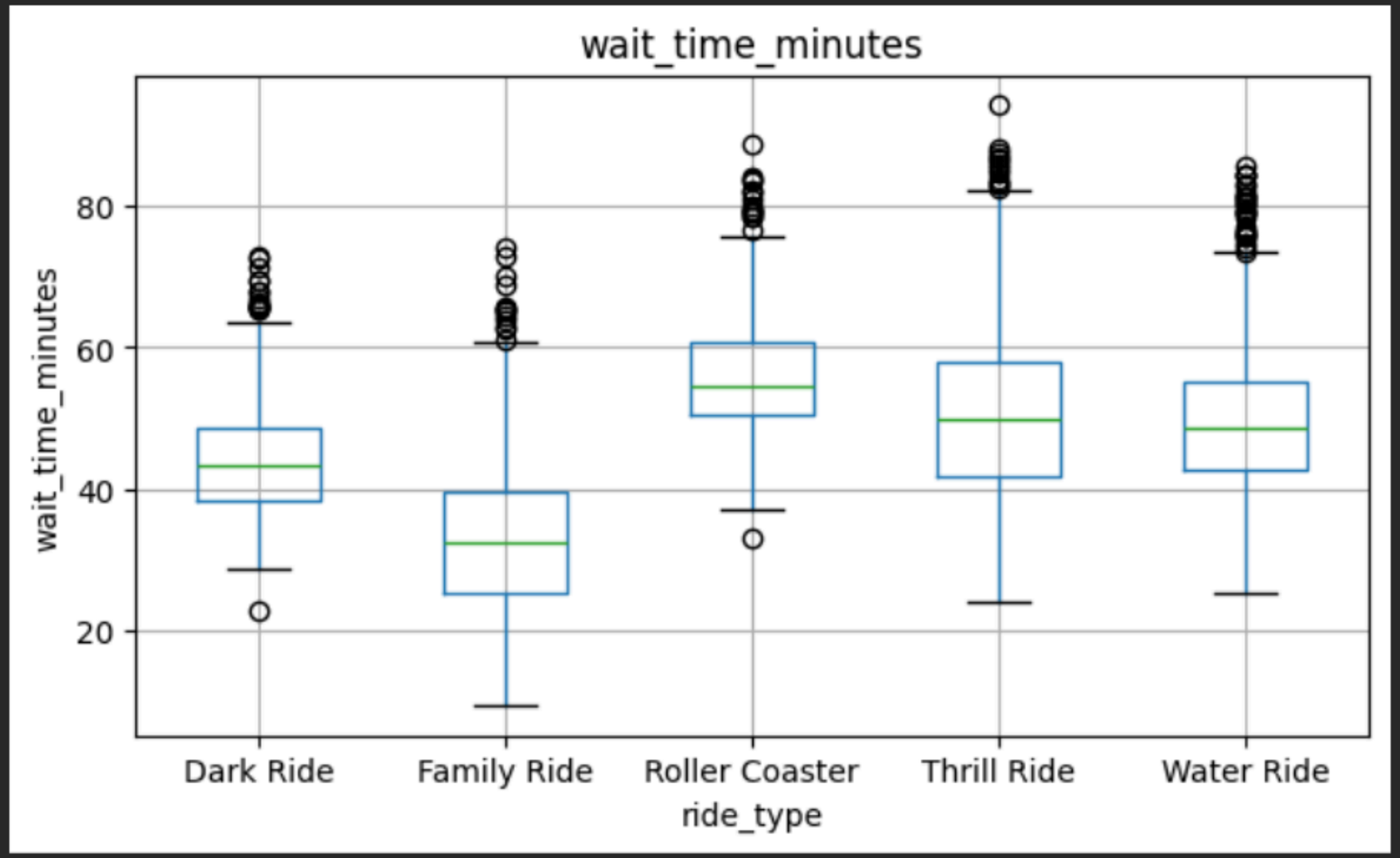
wait time based on hour of the day

```
hour = pd.to_datetime(df["timestamp"]).dt.hour
df.groupby(hour)["wait_time_minutes"].mean().plot(marker="o", figsize=(7, 4))
plt.ylabel("average wait (minutes)")
plt.xlabel("hour of day")
plt.title("Average wait by hour")
plt.show()
```



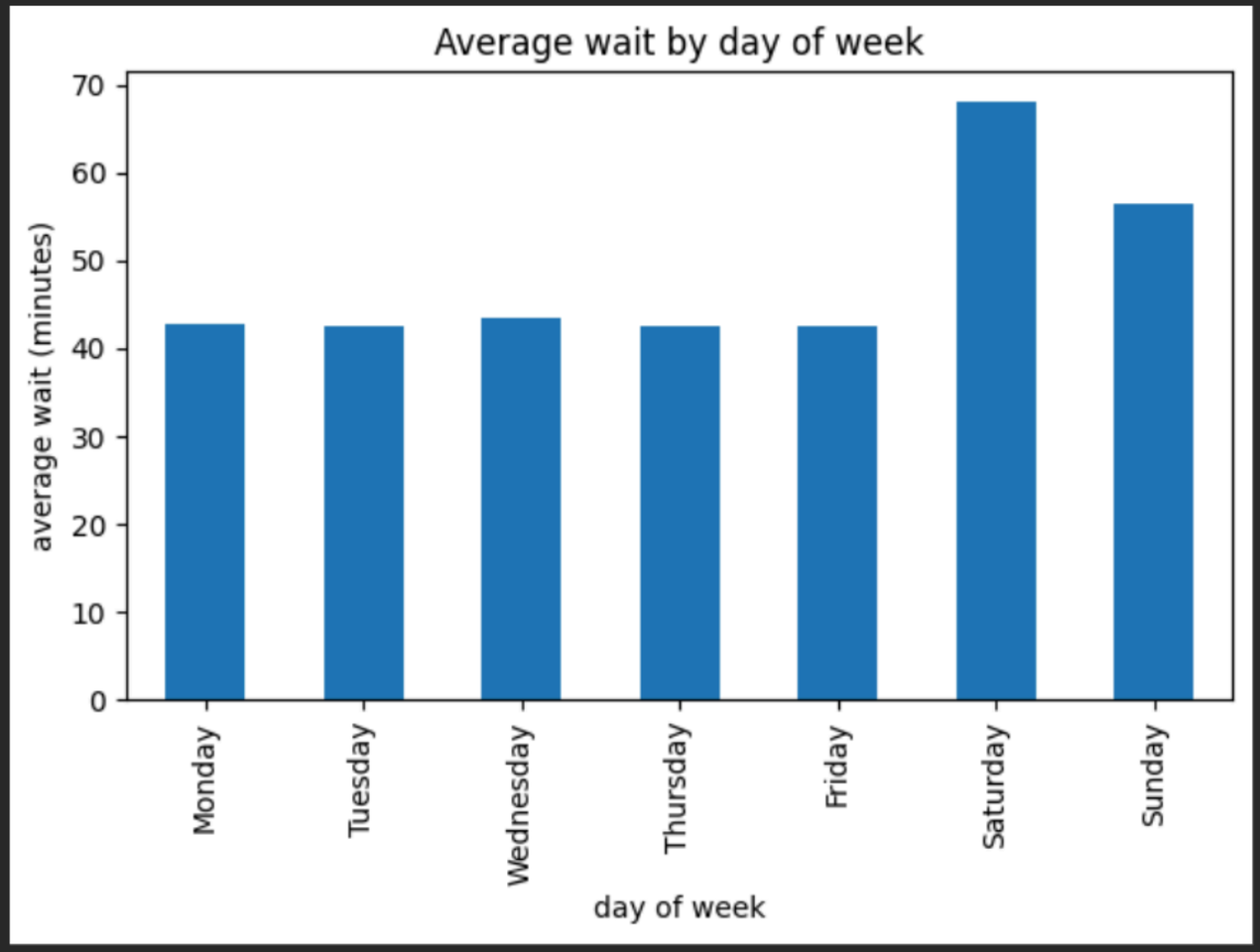
wait time based on ride type

```
df.boxplot(column="wait_time_minutes", by="ride_type", figsize=(7, 4))
plt.suptitle("")
plt.ylabel("wait_time_minutes")
plt.show()
```



Avg wait time on different days of the week

```
day = pd.to_datetime(df["timestamp"]).dt.day_name()
order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
df.groupby(day)["wait_time_minutes"].mean().reindex(order).plot(kind="bar", figsize=(7, 4))
plt.xlabel("day of week")
plt.ylabel("average wait (minutes)")
plt.title("Average wait by day of week")
plt.show()
```



```python
print(df.groupby("special_event")["wait_time_minutes"].mean().round(1))
print()
print(df.groupby(df["rainfall_mm"] > 0)["wait_time_minutes"].mean().round(1))
```

```
special_event
No     44.1
Yes    68.1
Name: wait_time_minutes, dtype: float64

rainfall_mm
False    46.7
True     43.4
Name: wait_time_minutes, dtype: float64
```

Rain pushes people indoors or out of the park entirely, dropping water ride demand. **special_event will be a massive predictive switch for our model.**

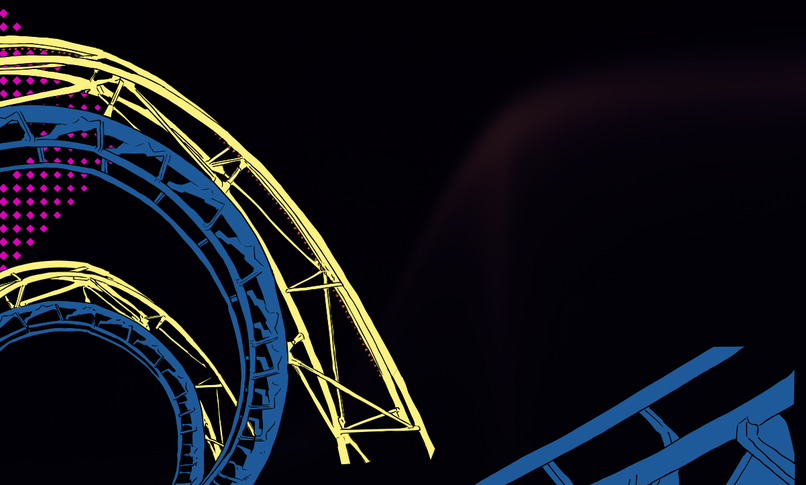
Massive positive relationship. What happened 30 minutes ago heavily dictates right now. This is our **star feature!**

Sharp negative relationship. High-capacity guest throughput actively destroys lines.

```python
# Which numeric columns move together with the target?
numeric_df = df.select_dtypes("number")
numeric_df.corr()["wait_time_minutes"].sort_values(ascending=False)
```

```
wait_time_minutes                1.000000
previous_wait_minutes            0.900850
park_crowd_index                 0.601460
ride_downtime_minutes_last_hour  0.179512
humidity_pct                    -0.005145
temperature_c                   -0.095088
rainfall_mm                     -0.131315
capacity_per_hour               -0.660031
Name: wait_time_minutes, dtype: float64
```

What was one core insight or relationship in the data that completely surprised you or went against your initial human guesses?



CLEANING AND PREPROCESSING THE DATA

```
[ 1. DateTime Sanitization ] ──────────> [ 2. Imputation Guardrail ]
 (Upgrading the Raw Text Dates)           (Plugging Potential Missing Holes)
```

The Problem:

Computers cannot natively perform equations on raw text strings like dates (e.g., "2026-09-01").

We use pd.to_datetime() to convert raw text into a live DateTime Object

The Problem:

Real-world datasets have empty rows or missing values (NaN entries).
If you feed an empty cell to a model in scikit-learn, your code will instantly crash.

We use .fillna() combined with the column's historical .median()
If any missing fields appear in the hidden test data, our pipeline automatically plugs those holes with a safe historical average without breaking.

TRAIN/TEST SPLIT

The problem: Our data is time-ordered, a random split will scatter rows from the same day or even half hour

The model could train on 2:00pm and be tested on 2:30pm, since the answers are nearly identical, it looks more smarter than it actually is

```python
# Train/validation split BY DATE, not randomly
def time_split(frame, val_days=2):
    """Hold out the most recent `val_days` days as validation."""
    days = frame["timestamp"].dt.normalize().drop_duplicates().sort_values()
    cutoff = days.iloc[-val_days]
    return frame[frame["timestamp"] < cutoff].copy(), frame[frame["timestamp"] >= cutoff].copy()

train_data, val_data = time_split(df_encoded)
```

training rows, validation rows and validation days (Note that there are only 2 days here)

```
train rows: 1728 | val rows: 384
validation days: ['Thursday', 'Friday']
```

The Fix: We split the dataset by date, not randomly

time_split() keeps the last 2 days for validation, and trains on everything before those days. So no rows appear in both sets of days.

FEATURE ENGINEERING

The idea: We give the model better clues by building new columns from the ones we already have

Time Features: The model can't read a timestamp, so we break it into hour, day of the week and a weekend flag

Cyclical Encoding: 11pm and midnight are neighbours, but as numbers (23 and 0) they look far apart. sin/cos wraps the hours into a circle, like a clock

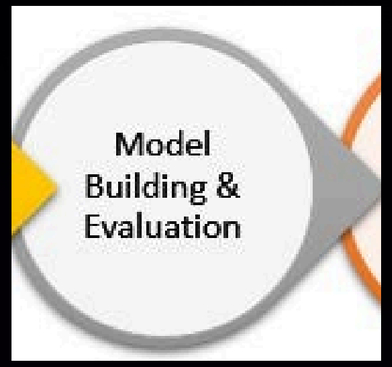
Peak hours: is_peak marks 1pm to 7pm, the busiest hours we found on our charts

```python
def make_features(frame):
    """Build all engineered features. Used for train AND test."""
    out = frame.copy()
    out["timestamp"] = pd.to_datetime(out["timestamp"])

    # --- time features ---
    out["hour"] = out["timestamp"].dt.hour + out["timestamp"].dt.minute / 60
    out["day_of_week"] = out["timestamp"].dt.dayofweek        # Monday=0 ... Sunday=6
    out["is_weekend"] = (out["day_of_week"] >= 5).astype(int)

    # cyclical encoding of the hour
    out["hour_sin"] = np.sin(2 * np.pi * out["hour"] / 24)
    out["hour_cos"] = np.cos(2 * np.pi * out["hour"] / 24)

    # busiest hours in our EDA were the afternoon
    out["is_peak"] = out["hour"].between(13, 19).astype(int)

    # interaction features
    out["crowd_per_capacity"] = out["park_crowd_index"] / out["capacity_per_hour"] * 1000
    out["weekend_x_crowd"] = out["is_weekend"] * out["park_crowd_index"]

    return out
```

Interaction features:

crowd_per_capacity: a big crowd at a low-capacity ride means a long line

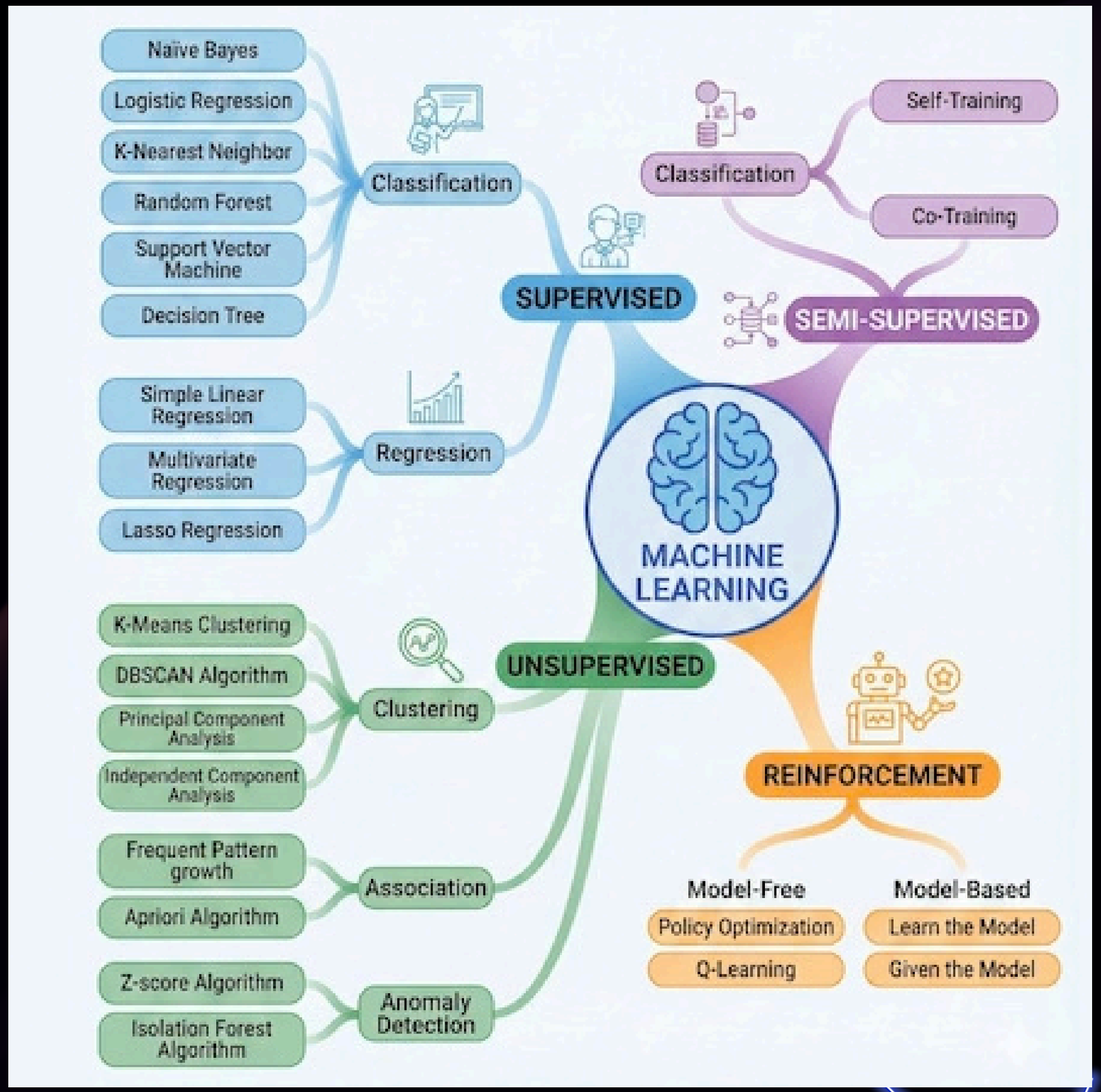
weekend_x_crowd: lets the model treat a weekend crowd differently from weekdays



STEP 2: BUILDING THE MODEL

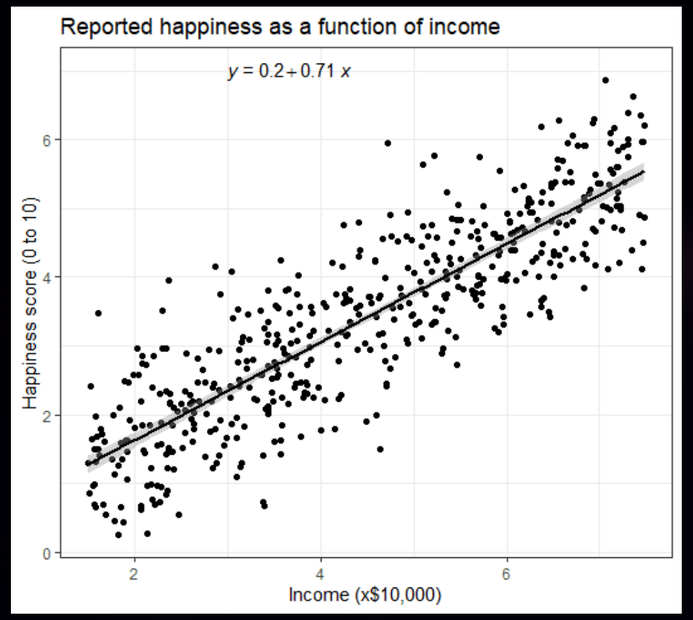
NOW THAT OUR DATA IS CLEAN AND FEATURE-RICH, LET'S TEACH A MACHINE TO FIND THE PATTERN.

MACHINE LEARNING MODELS

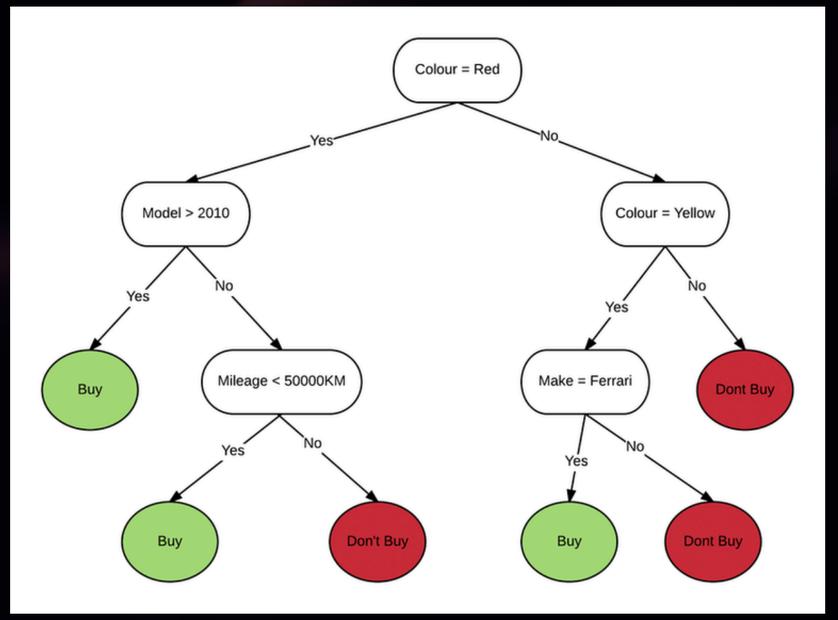


DIFFERENT MODELS, DIFFERENT APPROACHES

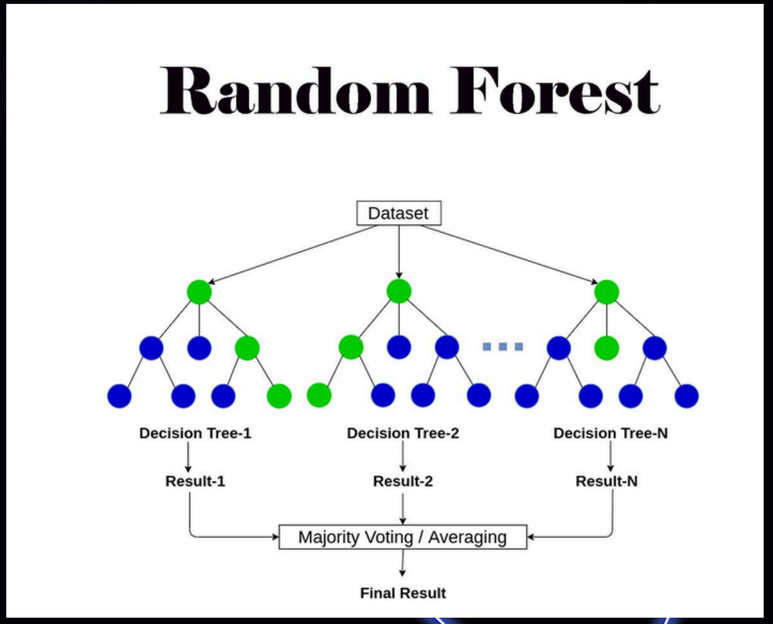
Linear Regression



Decision Trees



Random Forests



LINEAR REGRESSION

The idea: Find the best straight line through the data

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n$$

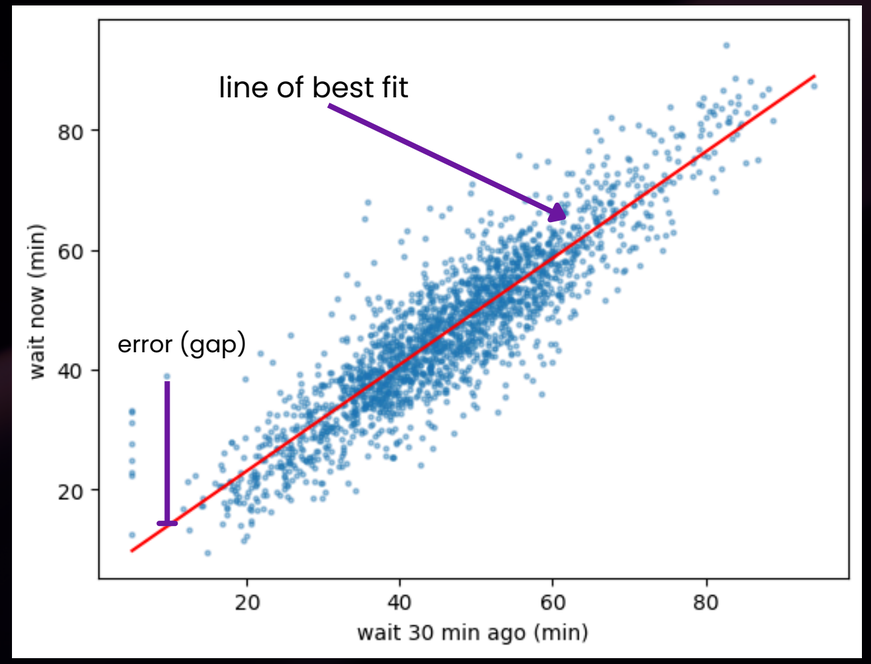
$\hat{y}$ = predicted wait, x = a feature, b = its weight, b_0 = starting value

Example

Start with: 10 min
It's the weekend → +5 min
There's a parade → +5 min
Predicted wait: 20 minutes

Each feature adds (or takes away) its own amount. Those amounts are the weights.

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 \rightarrow 10 + (5 \times 1) + (5 \times 1) = 20$$



Error = the gap between the actual wait (the dot) and the model's guess (the line)
The model picks the line that keeps the gaps as small as possible

Advantages

1. Simple and fast to train
2. Can work with smaller datasets

Disadvantages

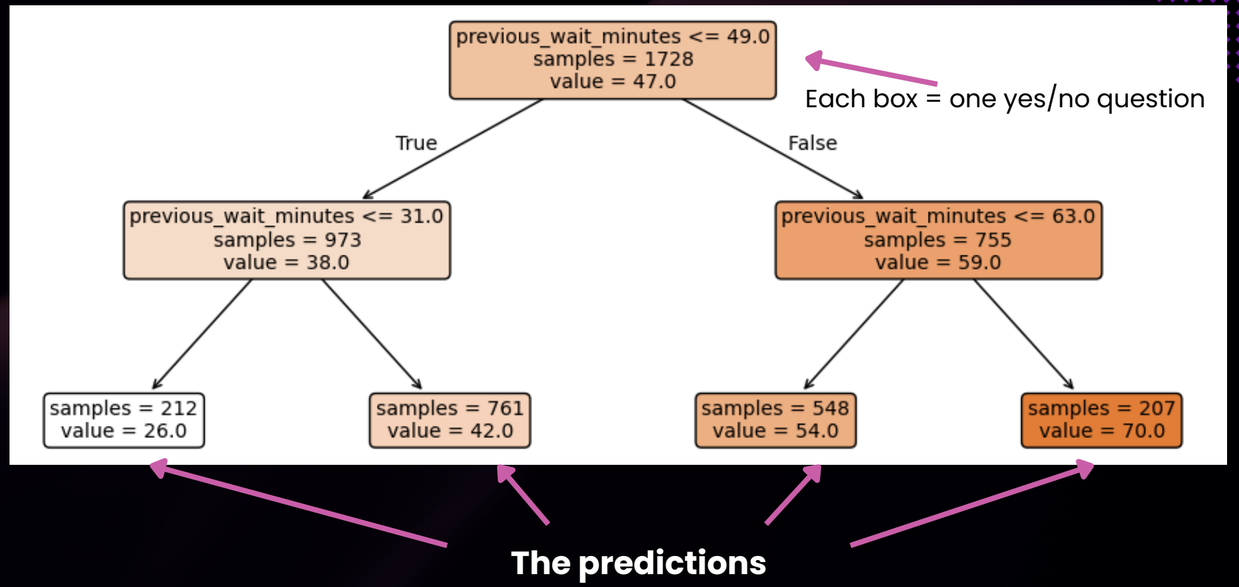
1. Cannot map patterns that curve, only straight line relationships
2. A few extreme values can tilt the whole line

DECISION TREES

The idea: A decision tree predicts by asking a series of yes/no questions about the data

How the tree is built

1. It starts with all the training rows in the top box
2. It then tests different conditions, and picks the questions that make it's predictions the most accurate
3. This repeats for each new group uptil a limit, called the maximum depth. Ex: A model can use 8 questions for an output



Example: Predicting a new row

The wait 30 mins ago was 55 mins

Is 55<=49? : NO, so go right
Is 55<=63?: YES, so go left
Lands in the leaf "value = 54"
Prediction: 54 minutes

Advantages

1. Unlike linear regression, it can map more complex patterns that cannot be represented by a line
2. Automatic Feature selection: The model finds the most important features and places emphasis on them

Disadvantages

1. Can't predict values outside the range of its data
2. Unstable: Sensitive to minor changes that can lead to an entirely different tree structure

RANDOM FOREST

The idea: build many decision trees and average their predictions

How it **works**

1. It builds many decision trees. Ours for example uses 180
2. Each tree is trained on a different random sample of the training rows
3. Since each tree sees different rows, each asks slightly different questions

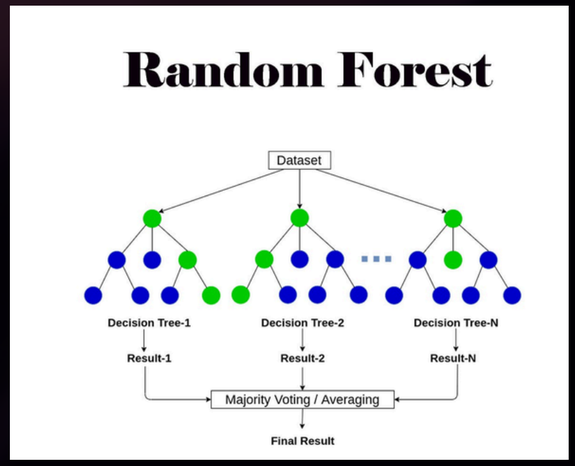
```
"Random Forest": RandomForestRegressor(
    n_estimators=180, max_depth=14, min_samples_leaf=2, random_state=42, n_jobs=-1
),
```

n_estimators = no of trees used(here, 180)

How it **predicts**

The random forest averages or takes the majority of the tree predictions

Example: Tree 1 - 42min, Tree 2 - 48min, Tree 3 - 45min
Predicted wait = (42+48+45)/3 = 45 mins



Advantages

1. More Stable: small changes in data -> predictions barely change -> more stable
2. More Accurate: Averaging many trees reduces the error any single tree might make

Disadvantages

1. Slower to train and predict: It has to build multiple trees and every prediction needs the input to pass through all of them
2. Hard to interpret: you can follow the decisions of a decision tree, but a forest averages many trees, so no single path explains its prediction.

APPLICATIONS OF THE MODELS

Linear Regression

1.House Prices

- Predict: House sale's price
- Using: Floor area, number of bedrooms, distance to nearest MRT
- Why Linear Regression?: each feature pushes the price up or down, the weights show by how much

2.Video Watch Time

- Predict: How many mins will a user watch a video
- Using: video length, user's avg watch time, time of day
- Why Linear Regression?: watch times tend to rise with these inputs, and the weights show which inputs matter the most

3.Sales from Advertising

- Predict: Monthly Sales
- Using: Money spent on TV, online and radio ads
- Why Linear Regression?: The weights would show how many extra sales each $1000 on each type of ad brings

Decision Tree

1.Loan Approvals

- Predict: Approve or Reject
- Using: Monthly income, existing debt, missed payments
- Why Decision tree?: The bank can show the exact conditions that led to a rejection

2.Emergency Doctors-Does a head injury need a CT scan?

- Predict: whether a child is at very low risk of serious brain injury, so they can skip the scan
- Using: symptoms like sever headaches, loss of consciousness, and how the injury happened
- Why Decision tree?:doctors can apply a short checklist in minutes and explain it, which spares children unnecessary radiation

In Use Today!: Emergency doctors use the PECARN rule that was built and tested using a decision tree, on data from 42,412 children in 25 US emergency departments .

APPLICATIONS OF THE MODELS

Random Forest

1. Mapping crops from satellite images

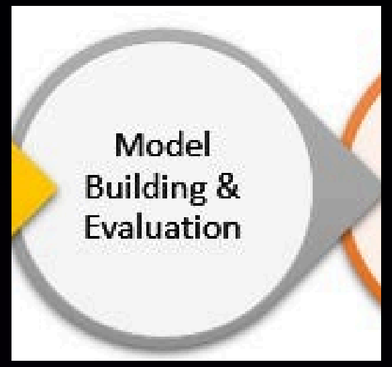
- Predict: which crop or land type covers each patch of land
- Using: satellite images from across the growing season, plus elevation
- Why a random forest: crops can look alike in one image but differ across the season, so it has to combine many inputs, and averaging trees reduces the effect of noisy pixels
- Real world use: the USDA switched its official US crop map from a decision tree to a random forest in 2024

2. Detecting online fraud

- Predict: whether a purchase is high risk
- Using: time, location and amount, compared with the customer's past activity
- Why a random forest: no single signal proves fraud; it's the combination that's suspicious, and averaging trees stops one odd detail from triggering a false alarm

3. Predicting where plants and animals can live

- Predict: whether an area is suitable for a species to live in
- Using: temperature, rainfall, elevation and slope
- Why a random forest: species respond to conditions in curved, combined ways (too hot or too cold, or only at certain heights), which a forest can capture
- Real world use: researchers funded by the US Forest Service used random forests to map where Douglas fir and subalpine fir grow, and projected their ranges to shrink under future warming

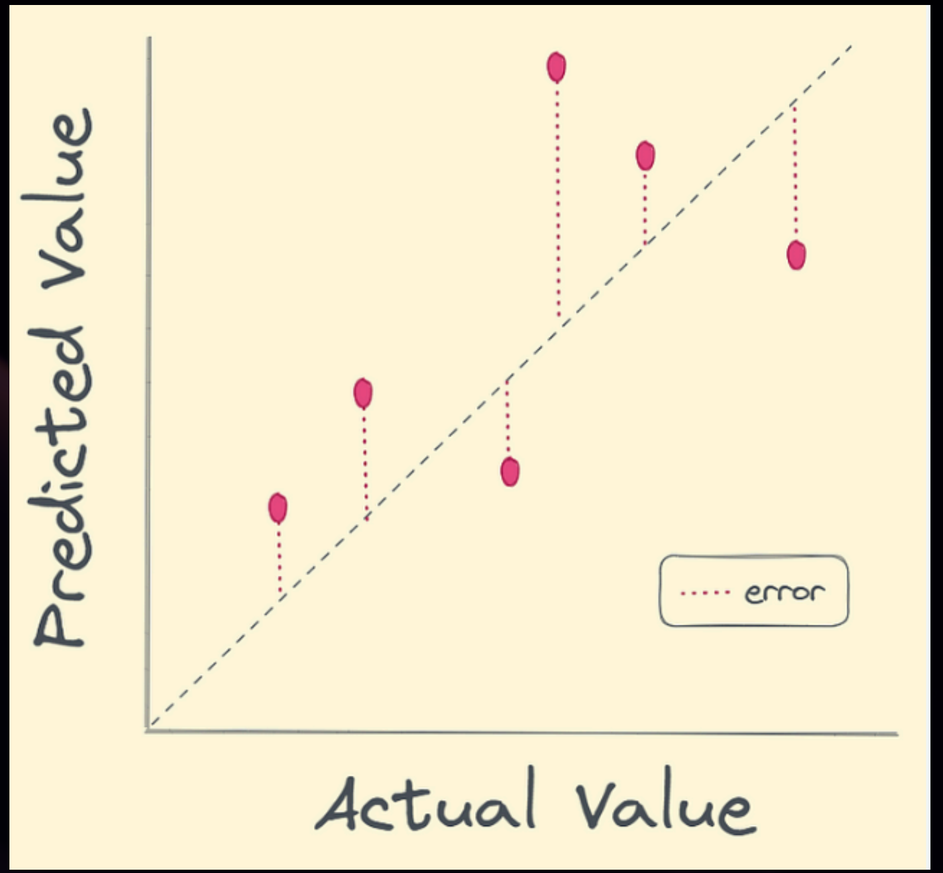


STEP 3 : EVALUATING OUR MODEL

NOW THAT OUR DATA IS CLEAN AND FEATURE-RICH, LET'S TEACH A MACHINE TO FIND THE PATTERN.

WHY WE NEED METRICS?

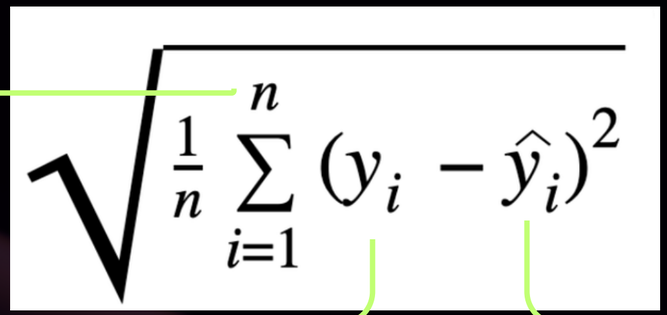
A model always outputs numbers, a list of raw predicted values. Metrics turn "here are some predictions" into "here's how wrong we typically are."



ROOT MEAN SQUARED ERROR (RMSE)

number of points considered

predicted value



actual value

Step by step: find each error, square it, average the squares, take the square root.

Why it's shaped this way:

- Squaring makes every error positive (so they can't cancel out) and **punishes big misses far harder** than small ones.
- The square root brings RMSE back into the same units as the target (e.g. bikes, dollars), so it's directly interpretable.

Find error ▶ Square it ▶ Average ▶ Square root

CODE APPLICATION

```
# PRE-REQUISITES ALREADY COMPLETED IN NOTEBOOK:
# 1. Feature Engineering complete on arrays
# 2. model.fit(X_train, y_train) has run successfully
# 3. pred = model.predict(X_val) has generated our validation guesses

from sklearn.metrics import mean_squared_error

# Step 1: Calculate the Mean Squared Error (Errors are squared first)
mse = mean_squared_error(y_val, pred)

# Step 2: Take the square root to convert the units back to minutes
rmse = mse ** 0.5

print(f"Validation RMSE: {rmse:.3f} minutes")
```

MEAN ABSOLUTE ERROR (MAE)

number of points considered

$$\frac{1}{N}\sum_{i=1}^{i=N}|y_i - \widehat{y_i}|$$

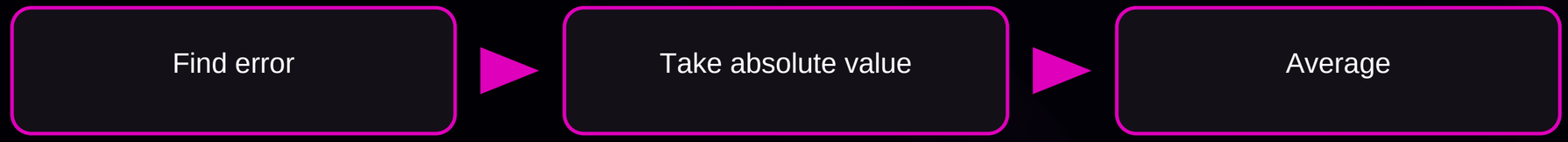
predicted value

actual value

Step by step: find each error, take its absolute value, average them.

Why it's shaped this way:

- No squaring, so every unit of error counts equally, a miss of 20 is exactly twice as bad as a miss of 10, not four times as bad.
- Also in the same units as the target, and easier to explain to a non-technical audience: "on average, we're off by X."



CODE APPLICATION

```
# PRE-REQUISITES ALREADY COMPLETED IN NOTEBOOK:
# 1. Feature Engineering complete on arrays
# 2. model.fit(X_train, y_train) has run successfully
# 3. pred = model.predict(X_val) has generated our validation guesses

from sklearn.metrics import mean_absolute_error

# Step 1: Calculate the linear average of absolute differences
mae = mean_absolute_error(y_val, pred)

print(f"Validation MAE: {mae:.3f} minutes")
```

UNDERSTANDING THE OUTPUT

Your Model's Output

- Is it > Baseline Guess? ——> [Bad Model: Code / Features Broken]
- Is it = Standard Model? ——> [Average Model: Basic Rules Learned]
- Is it < Standard Model? ——> [Winning Model: High Leaderboard Rank]

RMSE = 0: Perfect, flawless model (impossible on noisy real-world data).
MAE tells you your true, unpenalized average distance from the real answers.

READING RMSE & MAE TOGETHER

Case A

RMSE ≈ MAE

Your model's errors are small, consistent, and predictable. It makes minor mistakes uniformly across all rides.

Case B

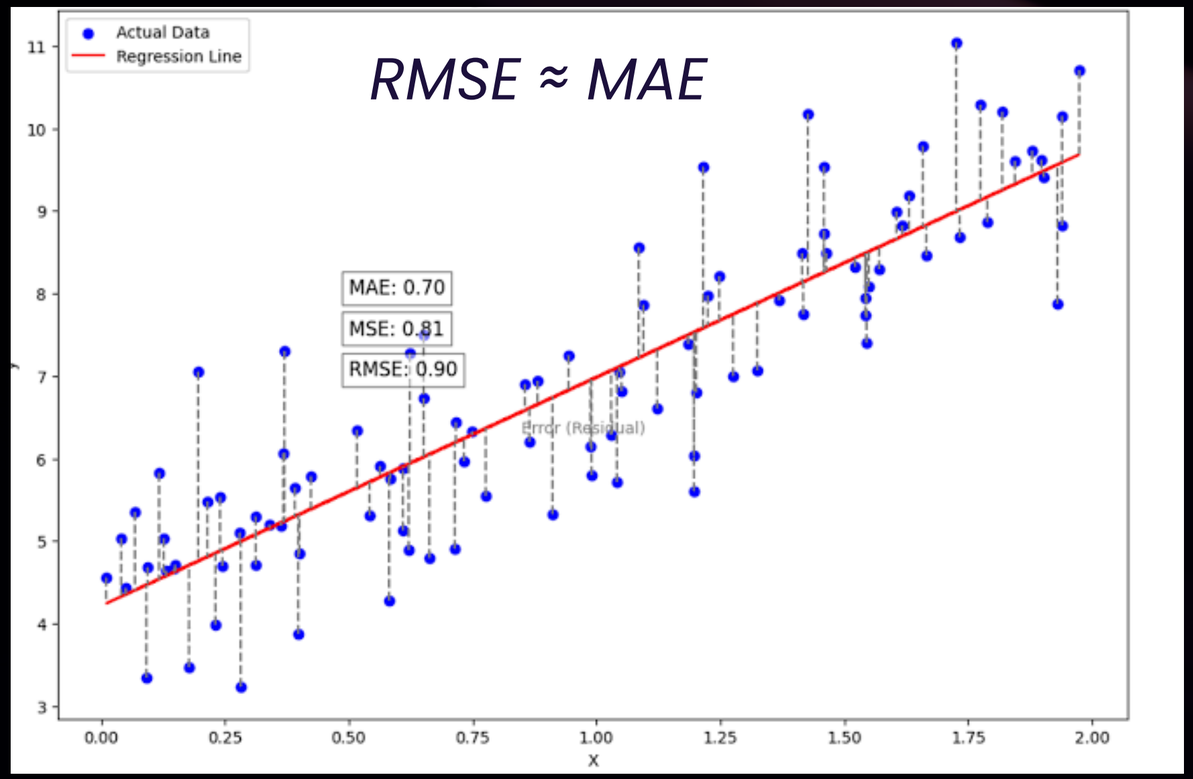
RMSE ≫ MAE

If your model shows a low MAE but a massive RMSE, it means your features work **great for normal park conditions.** However, your model is completely **blind to extreme situations (like ride failures or bad weather)**

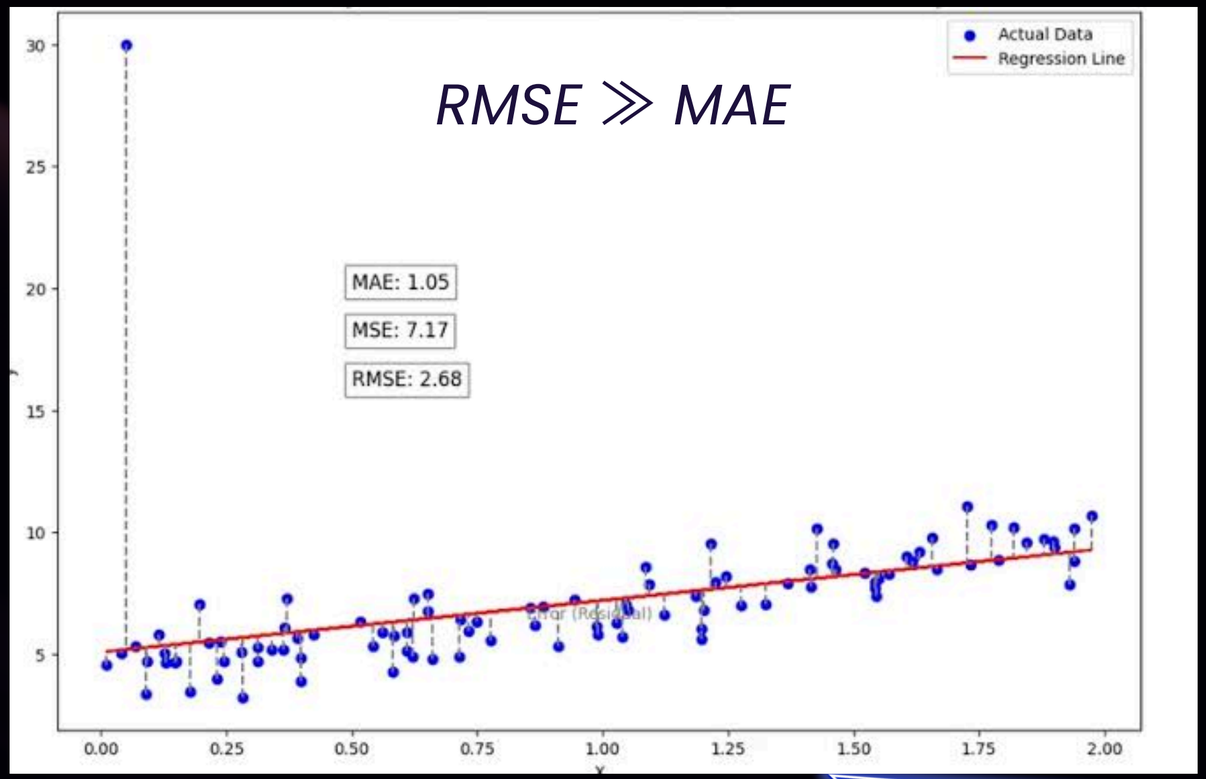
it is mathematically impossible for RMSE to be less than MAE.

READING RMSE & MAE TOGETHER

Case A



Case B



R^2 : COEFFICIENT OF DETERMINATION

Residual Sum of Squares

$$1 - \left(\frac{SSR}{SST}\right) = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y}_i)^2}$$

Total Sum of Squares

Predicted values from regression model

Mean of observed values

actual observed values

SSR The total squared error made by your machine learning model. It tracks how far your smart model's guesses land from the real numbers.

SST The total squared error made by a completely lazy baseline.
Imagine an empty script that doesn't look at weather, time, or ride names, it simply guesses the flat park average ($\bar{y}$) for every single row.

Step by step: how much error does your model have, versus how much error you'd have by just predicting the average every time?

Too much Math?

Let’s understand this using 2 different scenarios at the theme park!!

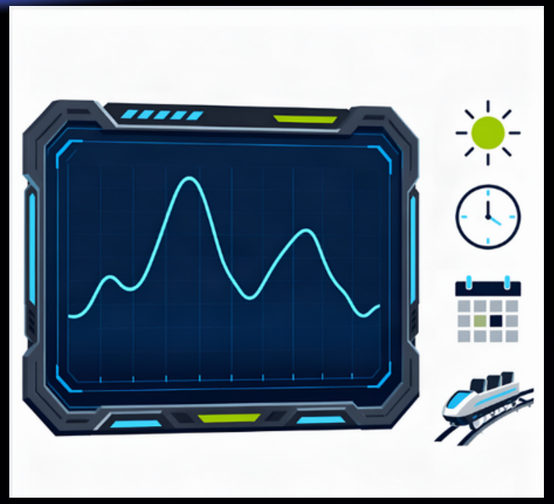
Imagine a lazy park operator who refuses to look at the time, check the weather, or even read the ride name. For every single prediction, they blindly guess the flat park **Average wait time: 35 minutes.**

A Lazy Baseline Guess

They get completely crushed by the data's chaos.

$$SST$$

GUESS	CONDITION	ACTUAL
35 MINS	PEAK 2:00PM CROWD	80 MINS
35 MINS	POURING RAIN	5 MINS

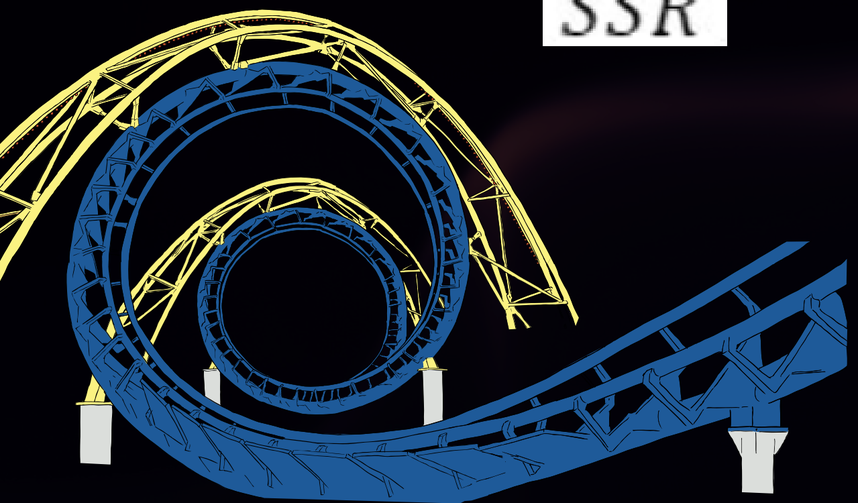


Your Machine Learning Model

SSR

Your algorithm is hyper-observant. It actively tracks that it’s a **Saturday afternoon**, **32°C outside**, a **Special Parade** is starting, and the attraction is a high-demand **Water Ride**.

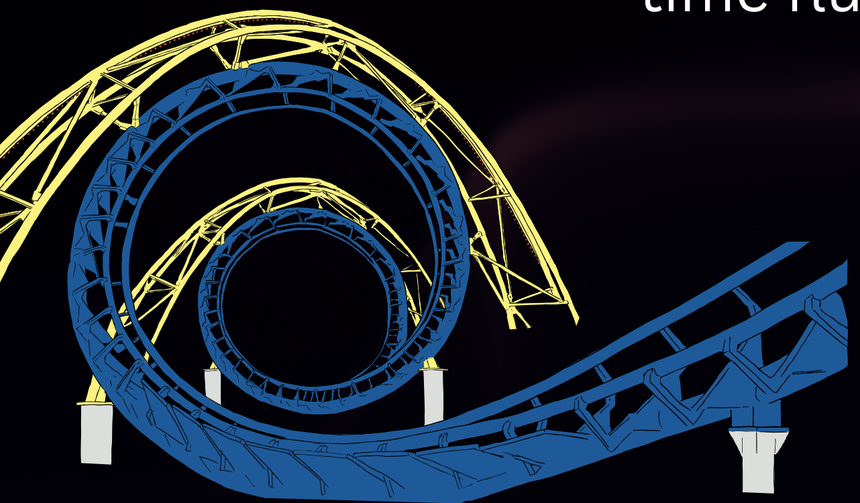
Because it understands the park's patterns, it dynamically scales its predictions up and down, tracking the real wait times almost perfectly.



SO WHAT EXACTLY DOES R^2 TELL US?

It measures the percentage of chaos explained.
If your model gets an R^2 score of 0.849,

It tells you that your creative feature engineering successfully captured and handled 84.9% of the wild wait-time fluctuations in the park. Only 15.1% remains unexplained chaos!

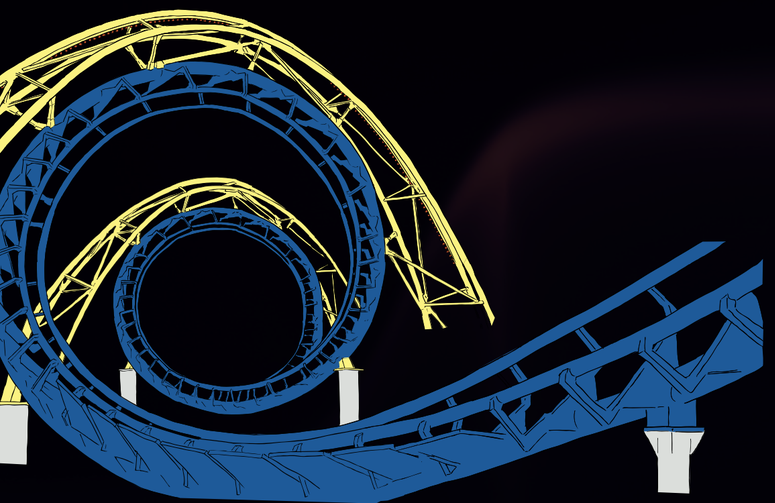


SO WHAT EXACTLY DOES R^2 TELL US?

$$1 - \left(\frac{SSR}{SST}\right)$$

what % of baseline mistakes your model still makes.

eg : 0.15
model suffers from 15% of the baseline mistakes



1 - 0.15 = 0.85

Your model successfully eliminated 85% of the chaos!

CODE APPLICATION

```
# PRE-REQUISITES ALREADY COMPLETED IN NOTEBOOK:
# 1. Feature Engineering complete on arrays
# 2. model.fit(X_train, y_train) has run successfully
# 3. pred = model.predict(X_val) has generated our validation guesses

from sklearn.metrics import r2_score

# Step 1: Calculate the R-squared Score
r2 = r2_score(y_val, pred)

print(f"Validation R2 Score: {r2:.3f}")
```

MULTIPLE METRICS TOGETHER

MAE tells you your "Normal Day" Performance

It gives you the pure, unpenalized average distance. It answers: *"On a typical, predictable afternoon at the park, how many minutes off is my guess?"*

RMSE catches your "Catastrophic Blunders"

It squares the errors before averaging them, heavily penalizing large misses. Answers: 'Is my model panicking and making catastrophic predictions during high-stress operational anomalies like rain or breakdowns?

R^2 Score gives you your "Leaderboard Grade"

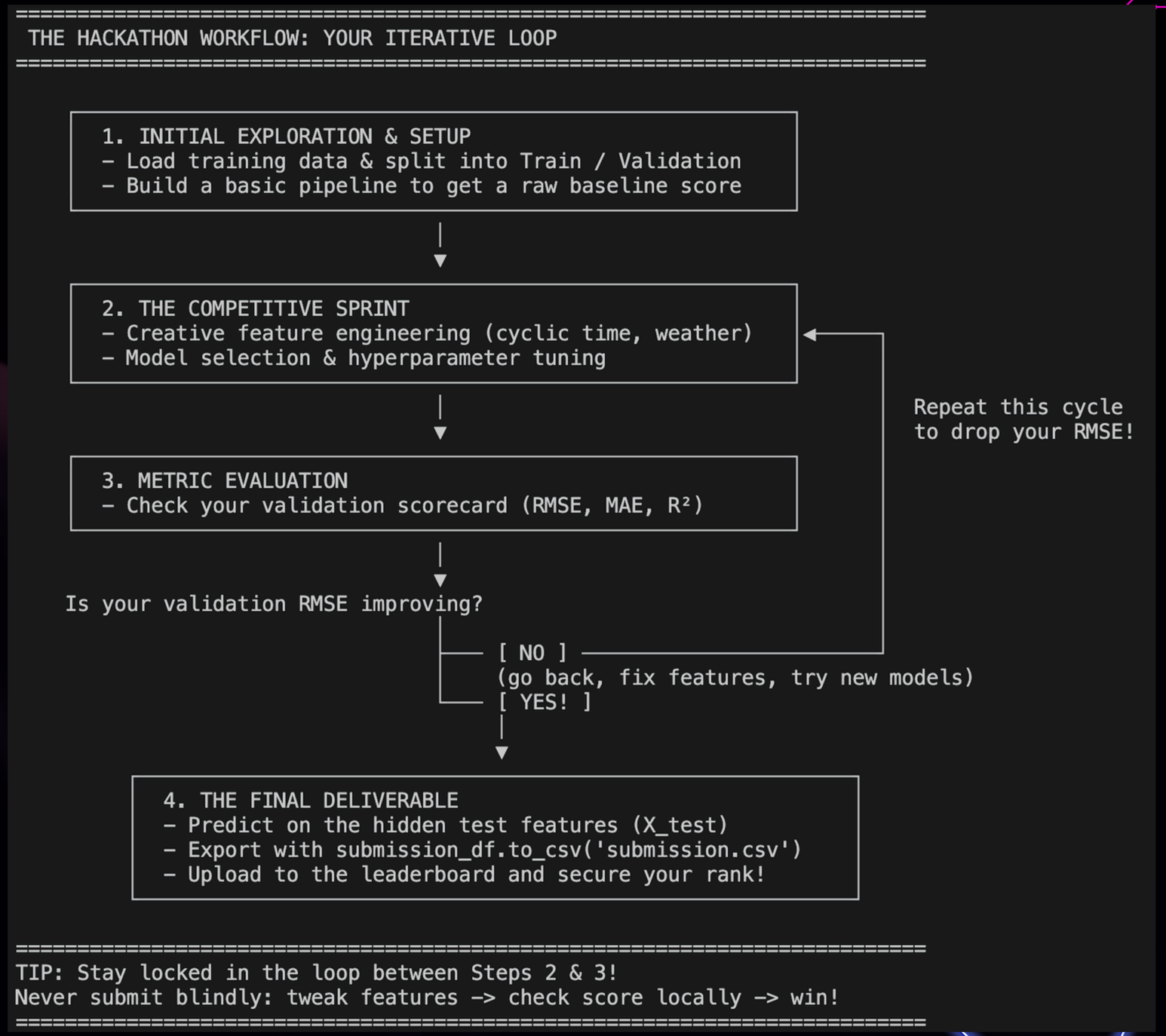
It translates those minute distances into a relative percentage. It answers: *"How much of the park's overall chaos did my feature engineering actually conquer compared to a completely lazy guess?"*

CODE APPLICATION

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def evaluate_hackathon_model(y_true, y_pred, model_name="My Model"):
    """Prints a unified diagnostic dashboard for regression models."""
    mae = mean_absolute_error(y_true, y_pred)
    rmse = mean_squared_error(y_true, y_pred) ** 0.5
    r2 = r2_score(y_true, y_pred)

    print(f"==========================================")
    print(f"      📊 {model_name.upper()} SCORECARD  ")
    print(f"==========================================")
    print(f"1. MAE  (Average Distance)  : {mae:.2f} minutes")
    print(f"2. RMSE (Extreme Miss Check): {rmse:.2f} minutes")
    print(f"3. R²   (Chaos Captured)    : {r2:.3f} ({r2*100:.1f}%)")
    print(f"==========================================\n")

# How to use it in your notebook after generating predictions:
evaluate_hackathon_model(y_val, pred, model_name="Linear Regression")
```

```
==========================================
      📊 LINEAR REGRESSION SCORECARD
==========================================
1. MAE  (Average Distance)  : 2.91 minutes
2. RMSE (Extreme Miss Check): 4.07 minutes
3. R²   (Chaos Captured)    : 0.849 (84.9%)
==========================================
```

SO THE ITERATIVE CYCLE IS



BEFORE WE LEAVE,

After executing the ML Pipeline, how has your initial guess changed?

Slido Code: 2502834

What do you think is the number one driving factor that causes a ride's wait time to skyrocket?

- A high overall park crowd density (park_crowd_index)
- Whether a parade or fireworks show is happening (special_event)
- What the queue looked like 30 minutes ago (previous_wait_minutes)
- The outdoor weather and temperature changing (temperature_c / rainfall_mm)

Your Canva profile name won't be shared

What do you think is the number one driving factor that causes a ride's wait time to skyrocket?

```
park_crowd_index                    0.601460
```

A high overall park crowd density (park_crowd_index)

75%

Whether a parade or fireworks show is happening (special_event)

6%

What the queue looked like 30 minutes ago (previous_wait_minutes)

17%

The outdoor weather and temperature changing (temperature_c / rainfall_mm)

2%

BEFORE WE LEAVE,

After executing the ML Pipeline, how has your initial guess changed?

Massive positive relationship. What happened 30 minutes ago heavily dictates right now. This is our **star feature!**

```
# Which numeric columns move together with the target?
numeric_df = df.select_dtypes("number")
numeric_df.corr()["wait_time_minutes"].sort_values(ascend

wait_time_minutes                1.000000
previous_wait_minutes            0.900850
park_crowd_index                 0.601460
```

What do you think is the number one driving factor that causes a ride's wait time to skyrocket?

- A high overall park crowd density (park_crowd_index)
- Whether a parade or fireworks show is happening (special_event)
- What the queue looked like 30 minutes ago (previous_wait_minutes)
- The outdoor weather and temperature changing (temperature_c / rainfall_mm)

Your Canva profile name won't be shared

ANY QUESTIONS?

SEE YOU AT THE DATATHON

ALL THE BEST!!